Marketing A/B Test Analysis: did the ad campaign increase conversions?

Setup: upload the Kaggle file (`marketing_AB.csv` or the JSON version) to Colab's Files panel, set the `FILE` path in the first code cell, then run the cells from top to bottom.

Tools: Python, pandas, scipy, matplotlib, seaborn

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

sns.set_theme(style="whitegrid")
FILE = "/marketing_AB.csv"        # use your file name; works for .csv and .json (if in /content use "/
```

1. Load And Clean

```
df = pd.read_csv(FILE)
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")
df = df.loc[:, ~df.columns.str.startswith("unnamed")]            # drop the index column if present
df["converted"] = df["converted"].astype(str).str.lower().isin(["true", "1", "yes"]).astype(int)
print(df.shape)
print(df.head(), "\n")
print("Missing values:\n", df.isna().sum(), "\n")
print("Duplicate user ids:", df["user_id"].duplicated().sum(), "\n")
```

```
(588101, 6)
   user_id test_group  converted  total_ads most_ads_day  most_ads_hour
0  1069124         ad          0        130       Monday             20
1  1119715         ad          0         93      Tuesday             22
2  1144181         ad          0         21      Tuesday             18
3  1435133         ad          0        355      Tuesday             10
4  1015700         ad          0        276       Friday             14

Missing values:
 user_id          0
test_group       0
converted        0
total_ads        0
most_ads_day     0
most_ads_hour    0
dtype: int64

Duplicate user ids: 0
```

2. The Experiment Setup

```
#    ad  = saw the real advertisement (treatment)
#    psa = saw a public service announcement (control / baseline)
# ------------------------------------------------------------------
summary = df.groupby("test_group")["converted"].agg(users="count", conversions="sum", conv_rate="mean")
summary["conv_rate_pct"] = (summary["conv_rate"] * 100).round(3)
print(summary, "\n")
```

```
print("Share of users in each group:\n", (df["test_group"].value_counts(normalize=True) * 100).round(1),
print("Note: the groups are very unequal in size. Say this in your write-up.\n")
```

```
             users  conversions  conv_rate  conv_rate_pct
test_group
ad          564577        14423   0.025547          2.555
psa          23524          420   0.017854          1.785

Share of users in each group:
 test_group
ad     96.0
psa     4.0
Name: proportion, dtype: float64

Note: the groups are very unequal in size. Say this in your write-up.
```

3. IS THE DIFFERENCE REAL? (two-proportion z-test + chi-square)

```
ad = summary.loc["ad"]; psa = summary.loc["psa"]
p_ad, p_psa = ad["conv_rate"], psa["conv_rate"]
n_ad, n_psa = ad["users"], psa["users"]

# z-test for a difference in proportions
p_pool = (ad["conversions"] + psa["conversions"]) / (n_ad + n_psa)
se_pool = np.sqrt(p_pool * (1 - p_pool) * (1 / n_ad + 1 / n_psa))
z = (p_ad - p_psa) / se_pool
p_value = 2 * (1 - stats.norm.cdf(abs(z)))

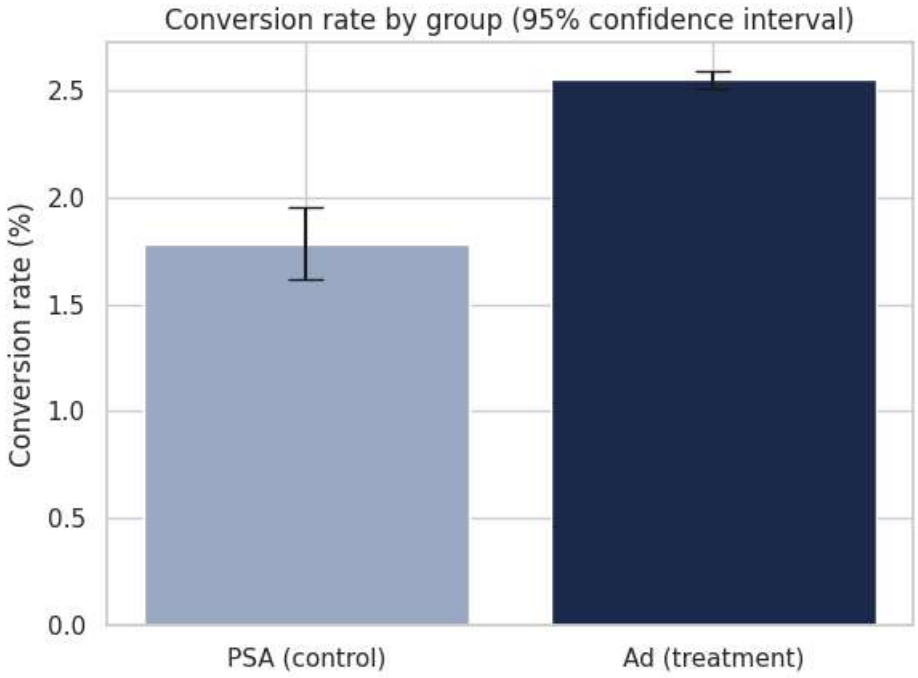
# 95% confidence interval for the difference
se_diff = np.sqrt(p_ad * (1 - p_ad) / n_ad + p_psa * (1 - p_psa) / n_psa)
diff = p_ad - p_psa
ci_low, ci_high = diff - 1.96 * se_diff, diff + 1.96 * se_diff

# chi-square test as a cross-check
chi2, p_chi, _, _ = stats.chi2_contingency(pd.crosstab(df["test_group"], df["converted"]))

print(f"Conversion rate  ad: {p_ad:.3%}   psa: {p_psa:.3%}")
print(f"Absolute lift: {diff*100:.2f} percentage points  (95% CI {ci_low*100:.2f} to {ci_high*100:.2f})")
print(f"Relative lift: {diff / p_psa:.1%}")
print(f"z = {z:.2f}, p-value = {p_value:.2e}   |   chi-square p-value = {p_chi:.2e}")
print("Statistically significant at 5%?", "YES" if p_value < 0.05 else "NO", "\n")

# chart: conversion rate with confidence intervals
fig, ax = plt.subplots(figsize=(6, 4.5))
rates = [p_psa * 100, p_ad * 100]
errs = [1.96 * np.sqrt(p_psa * (1 - p_psa) / n_psa) * 100, 1.96 * np.sqrt(p_ad * (1 - p_ad) / n_ad) * 1
ax.bar(["PSA (control)", "Ad (treatment)"], rates, yerr=errs, capsize=8, color=["#9aa8c4", "#1B2A4A"])
ax.set_ylabel("Conversion rate (%)")
ax.set_title("Conversion rate by group (95% confidence interval)")
plt.tight_layout(); plt.savefig("ab_conversion_by_group.png", dpi=150); plt.show()
```

```
Conversion rate  ad: 2.555%   psa: 1.785%
Absolute lift: 0.77 percentage points  (95% CI 0.60 to 0.94)
Relative lift: 43.1%
z = 7.37, p-value = 1.71e-13   |   chi-square p-value = 2.00e-13
Statistically significant at 5%? YES
```



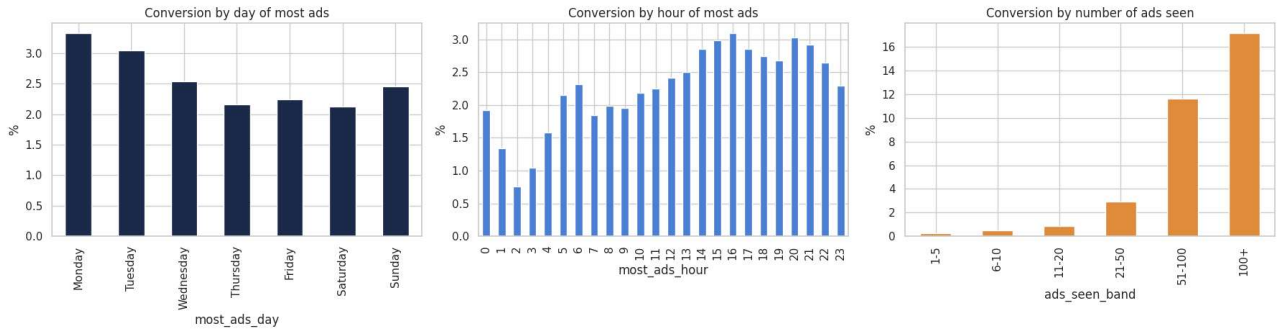
4. WHEN AND HOW MUCH? (only users who saw the ad)

```
ads = df[df["test_group"] == "ad"].copy()

order = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
by_day = (ads.groupby("most_ads_day")["converted"].mean() * 100).reindex(order)
by_hour = ads.groupby("most_ads_hour")["converted"].mean() * 100
ads["ads_seen_band"] = pd.cut(ads["total_ads"], bins=[0, 5, 10, 20, 50, 100, 5000],
                              labels=["1-5", "6-10", "11-20", "21-50", "51-100", "100+"])
by_band = ads.groupby("ads_seen_band", observed=True)["converted"].mean() * 100

fig, axes = plt.subplots(1, 3, figsize=(18, 4.8))
by_day.plot(kind="bar", ax=axes[0], color="#1B2A4A"); axes[0].set_title("Conversion by day of most ads");
by_hour.plot(kind="bar", ax=axes[1], color="#4a7fd4"); axes[1].set_title("Conversion by hour of most ads"
by_band.plot(kind="bar", ax=axes[2], color="#e08a3c"); axes[2].set_title("Conversion by number of ads see
plt.tight_layout(); plt.savefig("ab_segments.png", dpi=150); plt.show()

print("Best day:", by_day.idxmax(), f"({by_day.max():.2f}%)   Worst day:", by_day.idxmin(), f"({by_day.mi
print("Best hour:", by_hour.idxmax(), f"({by_hour.max():.2f}%)")
print(by_band.round(2).to_string(), "\n")
print("Caution: users who were already likely to convert may also see more ads, so the ads-seen pattern s
```




```
Best day: Monday (3.32%)   Worst day: Saturday (2.13%)
Best hour: 16 (3.09%)
ads_seen_band
1-5         0.25
6-10        0.49
11-20       0.84
21-50       2.92
```

5. Business Impact

```
# ASSUMPTIONS (change these to your own scenario)
campaign_users = 1_000_000          # people who will see the campaign
value_per_conversion = 50           # revenue per converted customer
campaign_cost = 100_000             # total campaign cost

extra_conversions = campaign_users * diff
extra_revenue = extra_conversions * value_per_conversion
print(f"Extra conversions per {campaign_users:,} users: {extra_conversions:,.0f}")
print(f"Extra revenue at {value_per_conversion} each: {extra_revenue:,.0f}")
print(f"Campaign cost: {campaign_cost:,.0f}   ROI: {(extra_revenue - campaign_cost) / campaign_cost:.0%}")
print(f"Break-even needs at least {campaign_cost / value_per_conversion / campaign_users * 100:.2f} perce
```

```
Extra conversions per 1,000,000 users: 7,692
Extra revenue at 50 each: 384,623
Campaign cost: 100,000   ROI: 285%
Break-even needs at least 0.20 percentage points of lift.
```

6. RECOMMENDATIONS (write these from YOUR results above)

```
Start coding or generate with AI.
```

1. Roll out / do not roll out the campaign, and why (lift, CI, p-value).

```
# 2. Best timing: [day / hour with the highest conversion].
# 3. Frequency: [does conversion keep rising with more ads, or flatten?].
# 4. Next test: [e.g. equal-sized groups, test different creatives, add revenue per user].
# LIMITATIONS: very unequal group sizes; association not proof for the ads-seen pattern;
# conversion is a yes/no outcome so it says nothing about revenue per customer.
```